

QuizMaster App

Student – Details

Name - A. Ganga Praneetha

Roll No – 23f1001883

Email – 23f1001883@ds.study.iitm.ac.in

Description

Aim of project is to build a Quiz Application backend using Flask, with role-based access (Admin/User). Implement APIs for subject, chapter, question, quiz management, user authentication, scoring, and exporting results.

Used AI/LLM (ChatGPT - GPT-4o) for around 40 % of the project. Heavily leveraged for frontend quiz timer logic, chart visualizations, OpenAPI YAML generation, and structuring backend API routes.

Technologies Used

Technology	Purpose
Flask	Core backend web framework used to build REST APIs
Flask-RESTful	Simplified API route creation and resource structuring
Flask-SQLAlchemy	ORM for managing database models and queries
Flask-Security	User authentication, authorization, and role-based access control
Celery	For handling background tasks like CSV report generation
Vue 3 (Composition API)	Frontend framework for creating a dynamic and reactive UI
Chart.js	To visualize user quiz performance with interactive charts
SQLite /	Database for storing users, quizzes, questions, and scores
YAML (OpenAPI)	To define and document RESTful API structure
Jinja2	Template engine for rendering HTML views (like index.html)

DataBase Schema The schema follows a normalized structure to maintain clear relationships between Subjects, Chapters, Questions, and Quizzes. This modular design ensures scalability, reduces redundancy, and makes it easier to manage quiz content dynamically.

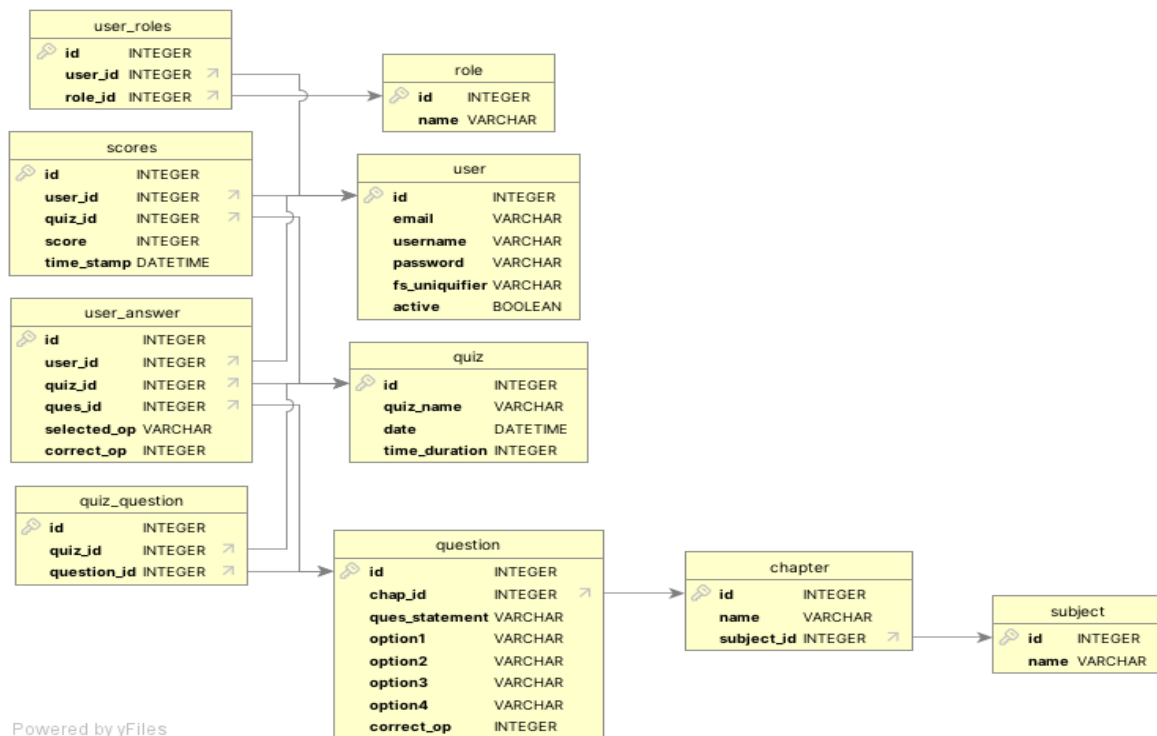


Figure 1: DatabaseSchema

API Design

The project includes RESTful APIs for managing Subjects, Chapters, Questions, Quizzes, Quiz Questions, User Answers, and Scores. Admins can create, update, and delete resources, while users can attempt quizzes and view scores. APIs are protected using Flask-Security with role-based access control. The APIs are implemented using Flask-Restful, with endpoints mapped to resource classes for clean separation of logic. The OpenAPI YAML file describes all endpoints, methods, parameters, and responses.

Architecture and Features

The project is organized as a modular Flask application. Controllers (routes and resources) are defined in the resources/ folder using Flask-Restful. The database models reside in the

model.py file, and the database.py handles initialization via Flask-SQLAlchemy.

Authentication and role management are handled using Flask-Security. Templates such as index.html are stored in the templates/ folder, and static CSV reports and javascripts files are served from the static/ directory.

Vedio link: https://drive.google.com/file/d/1Q6FnnXwctIPCup6S-OZ3jFTzyk7QmcZ6/view?usp=drive_link